TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 15 · LÝ THUYẾT

Đóng gói, Chuẩn bị triển khai và Bảo mật ứng dụng

Ba mức tái lập · docker-compose hoàn chỉnh · Biến môi trường và secret
Deployment Guide · Checklist bảo mật · Buổi lý thuyết cuối trước hạn nộp Bài tập 3

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

Nội dung

01 **Đóng gói tái lập được — trọng tâm của buổi**
Ba mức tái lập; Dockerfile và docker-compose hoàn chỉnh có healthcheck và dữ liệu mẫu; phép thử máy trắng; phương án thay thế khi không dùng được Docker.

02 **Biến môi trường và secret**
Vì sao tách cấu hình khỏi mã; .env, .env.example, .gitignore, .dockerignore; bốn nơi không được để secret; xử lý khi lỡ lộ.

03 **Deployment Guide: viết cho ai và viết gì**
Năm mục bắt buộc của đề BT3, viết bằng tiếng Anh; mục “cách kiểm tra hệ thống đã chạy đúng”.

04 **Checklist bảo mật cơ bản**
OWASP Top 10:2025 ở mức nhận biết; bảy mục checklist BT3 và cách tự kiểm từng mục bằng lệnh có minh chứng.

05 **Tài liệu kỹ thuật theo track và nộp Bài tập 3**
API docs [SE] · Pipeline doc + Data dictionary [DA] · Model card [AI]; nội dung nộp, điểm cộng deploy thử.

Mục tiêu

1 Đóng gói tái lập được

- Viết Dockerfile và docker-compose có healthcheck, dữ liệu mẫu
- Tự chạy phép thử máy trắng và sửa lỗi phát hiện được

2 Quản lý cấu hình và secret

- Tách cấu hình ra biến môi trường; .env.example đủ biến
- Xử lý đúng khi lỡ lộ secret

3 Viết tài liệu bàn giao

- Deployment Guide năm mục bằng tiếng Anh
- Tài liệu kỹ thuật theo track mô tả đúng hệ thống thật

4 Tự đánh giá bảo mật

- Nhận biết OWASP Top 10:2025
- Điền checklist bảy mục, mỗi mục có minh chứng chạy thật

Bốn mục tiêu này là đầu vào trực tiếp cho thành phần 4, 5, 6 của Bài tập 3 — hoàn thiện ở workshop buổi 16 và nộp trước khi kết thúc buổi 16.

ĐẶT VẤN ĐỀ

“Chạy được trên máy tôi” chưa phải là sản phẩm. Sản phẩm là thứ NGƯỜI KHÁC dựng lại được.

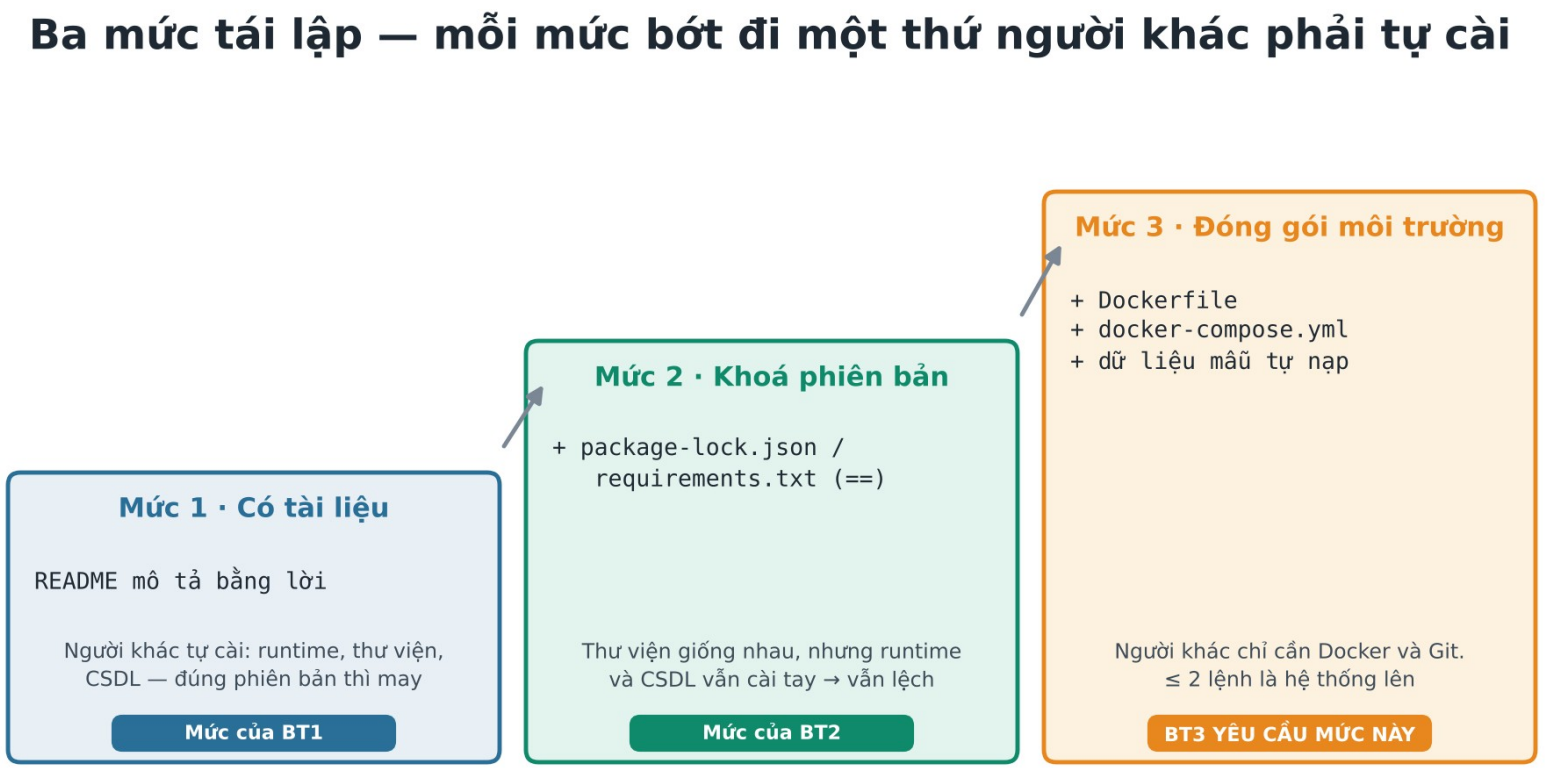
- Ở buổi 16, giảng viên clone repo của sinh viên theo tag v1.0 vào **thư mục trống**, làm đúng theo Deployment Guide và **không hỏi sinh viên câu nào**. Hệ thống lên được thì đạt mục CỐT LÕI C1.
- C1 là mục quan trọng nhất của BT3: một gói không dựng lại được thì **mọi test case và tài liệu đều không kiểm chứng được** — người chấm không có gì để chạy.
- Máy sinh viên có những thứ máy người khác không có: Node/Python đúng phiên bản, CSDL đã có bảng và dữ liệu, biến môi trường đặt từ tuần trước. Đóng gói là **đưa tất cả những thứ ngầm đó vào repo**.
- Bảo mật ở học phần này **không phải kiểm thử xâm nhập** — mà là bảy việc cơ bản sinh viên tự kiểm, ghi trung thực, có minh chứng.

01

Đóng gói tái lập được

Ba mức tái lập · Dockerfile và compose hoàn chỉnh · phép thử máy trắng · phương án thay thế

Từ “trên máy tôi chạy được” tới “máy nào cũng chạy được”



Ví dụ vì sao mức 2 chưa đủ: khoá đúng pandas nhưng máy người chấm có Python 3.9, mã dùng cú pháp 3.11 → vẫn vỡ.

Đọc hình thế nào

- Mức 1 (README bằng lời) là mức BT1; mức 2 (thêm tệp khoá phiên bản) là mức BT2.
- Mức 2 vẫn lệch: runtime và CSDL vẫn phải cài tay, mỗi máy một phiên bản.
- Mức 3: container chứa runtime, thư viện, cấu hình; người khác chỉ cần Docker.
- Chỉ có README bằng lời → **KHÔNG đạt** thành phần 4 của BT3.

Cấu trúc một gói đóng gói hoàn chỉnh

Từ một máy chỉ có Docker và Git — không cài Node, Python hay PostgreSQL

```
# ===== Người chấm làm đúng hai lệnh này (sau khi clone tag v1.0) =====
cp .env.example .env
docker compose up -d --build

# ===== Những gì repo phải có để hai lệnh trên chạy được =====
Dockerfile                  # đóng gói ứng dụng
docker-compose.yml          # app + CSDL (+ model [AI]), có healthcheck
.dockerignore               # không chép node_modules, .venv, .env
.env.example                # ĐỦ tên biến, giá trị MẪU chạy được
db/init/                    # Postgres tự chạy khi volume còn trống
  001_create_tables.sql
  002_seed_danh_muc.sql     # trung tâm, kỹ thuật viên, khách, phiếu mẫu
docs/deployment-guide.md    # tiếng Anh, năm mục, 1–2 trang

# ===== Phép thử trước khi nộp — bắt buộc tự làm =====
docker compose down -v      # xoá sạch volume, giả lập máy trắng
docker compose up -d --build
docker compose ps           # mọi dịch vụ "healthy"; /health -> ok
```

Ghi nhớ

- Postgres **không đọc thư mục con** của docker-entrypoint-initdb.d — để mọi tệp .sql ngang hàng trong db/init/.
- Thiếu **dữ liệu mẫu**: hệ thống lên được nhưng không thao tác được gì — lỗi hay gặp nhất.
- Bước khởi tạo (migrate, tạo tài khoản demo) phải **tự động** — không bắt người chấm gõ lệnh thứ ba.
- Endpoint /health là cách nhanh nhất để người chấm biết hệ thống đã lên.

Dockerfile cho dịch vụ Node và dịch vụ Python

Dockerfile — API Node.js [SE]

```
FROM node:20-alpine
WORKDIR /app

# 1. Tệp khoá trước -> chỉ cài lại khi
#    package-lock.json đổi (tận dụng cache)
COPY package.json package-lock.json ./
# BT3 chạy test TRONG container nên cài
# cả devDependencies (jest, supertest)
RUN npm ci

# 2. Mã nguồn và test
COPY src ./src
COPY tests ./tests

# 3. Không chạy bằng root
USER node
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Dockerfile — API mô hình / pipeline Python [AI][DA]

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY src ./src
COPY data/mau ./data/mau
# [AI] Tệp mô hình KHÔNG commit (.gitignore):
#      huấn luyện lại khi build, cố định seed
RUN python -m src.train --data data/mau \
      --seed 42 --out models/

RUN useradd -m appuser && chown -R appuser /app
USER appuser
EXPOSE 8000
CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port",
"8000"]
```

Thứ tự COPY quyết định tốc độ build: chép tệp khoá và cài thư viện trước, chép mã sau — sửa mã không phải cài lại thư viện.

Huấn luyện khi build với dữ liệu mẫu nhỏ và seed cố định: không commit tệp mô hình mà người chấm vẫn có mô hình chạy được.

DOCKER-COMPOSE HOÀN CHỈNH – THÀNH PHẦN 4 CỦA BT3

docker-compose.yml: CSDL đóng, app mở, chờ nhau đúng cách

docker-compose.yml — track SE (DA/AI thêm hoặc thay một dịch vụ, xem hình tiếp theo)

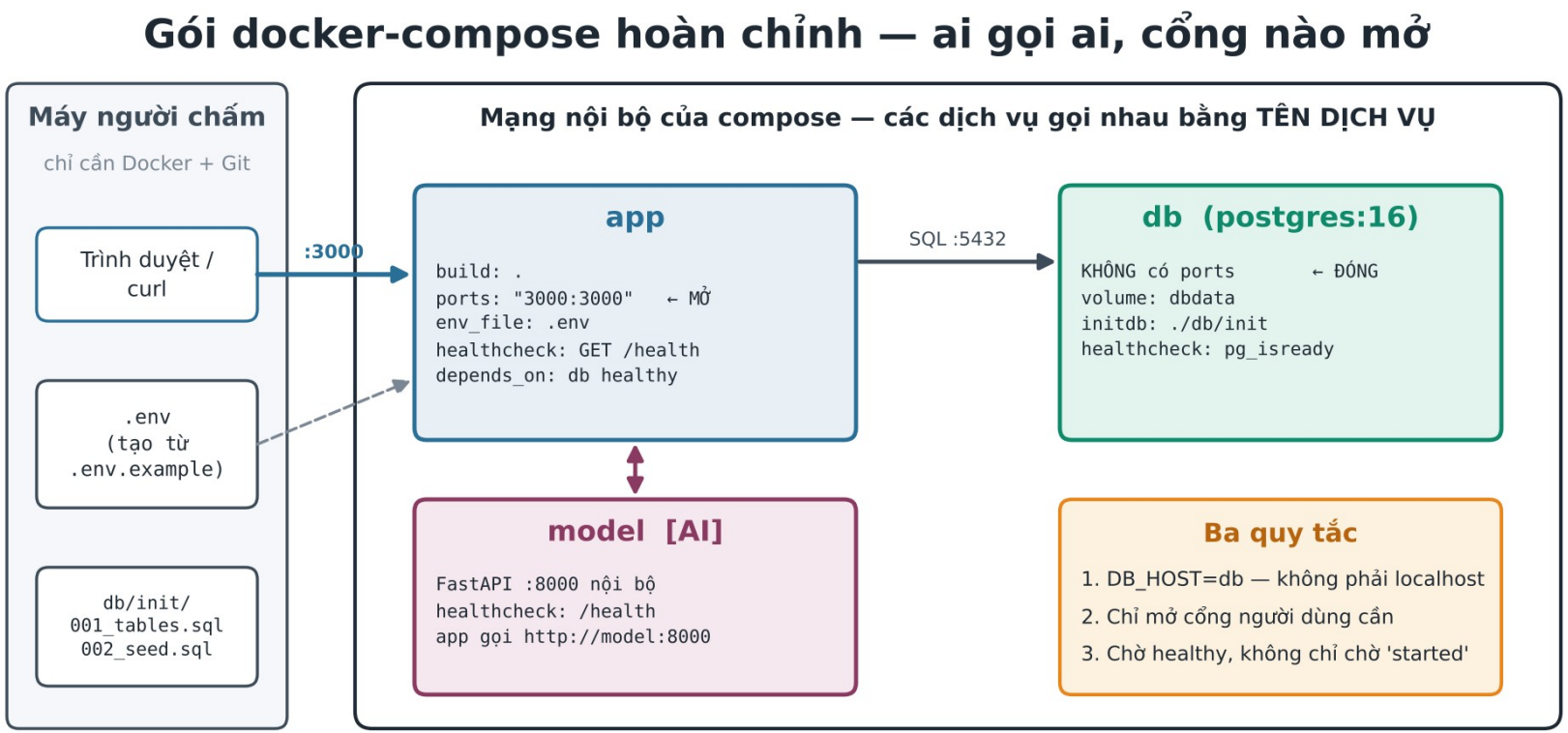
```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}          # lấy từ .env — không ghi cứng
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - dbdata:/var/lib/postgresql/data
      - ./db/init:/docker-entrypoint-initdb.d:ro   # bảng + dữ liệu mẫu
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 5s
      retries: 10
    # (không có ports: CSDL chỉ mở trong mạng nội bộ)
  app:
    build: .
    env_file: .env                       # mọi biến của ứng dụng
    environment:
      DB_HOST: db                        # TÊN DỊCH VỤ, không phải localhost
    ports:
      - "${APP_PORT:-3000}:3000"
    depends_on: { db: { condition: service_healthy } }
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:3000/health"]
volumes: { dbdata: {} }
```

Ghi nhớ

- **Không mở cổng CSDL** ra máy chủ khi không cần — tránh cấu hình sai (OWASP A02).
- **condition: service_healthy** — app chờ CSDL SẴN SÀNG, không chỉ chờ container khởi động.
- **healthcheck của app** cho người chấm thấy “healthy” trong docker compose ps.
- ${APP_PORT:-3000}: cổng 3000 bị chiếm thì đổi trong .env, không sửa compose.

MINH HOẠ — KIẾN TRÚC GÓI ĐÓNG GÓI

Ai gọi ai, cổng nào mở ra ngoài



Track DA: thay model bằng pipeline (docker compose run --rm pipeline) · Track SE không cần dịch vụ model.

Đọc hình thế nào

- Người dùng chỉ đi vào qua **một cổng** của app.
- Các dịch vụ gọi nhau bằng **tên dịch vụ** (db, model) trong mạng nội bộ.
- [AI] app gọi model qua http://model:8000; model hỏng thì app vẫn tạo phiếu được (suy giảm có kiểm soát, buổi 11).
- [DA] thay model bằng dịch vụ pipeline chạy một lần.

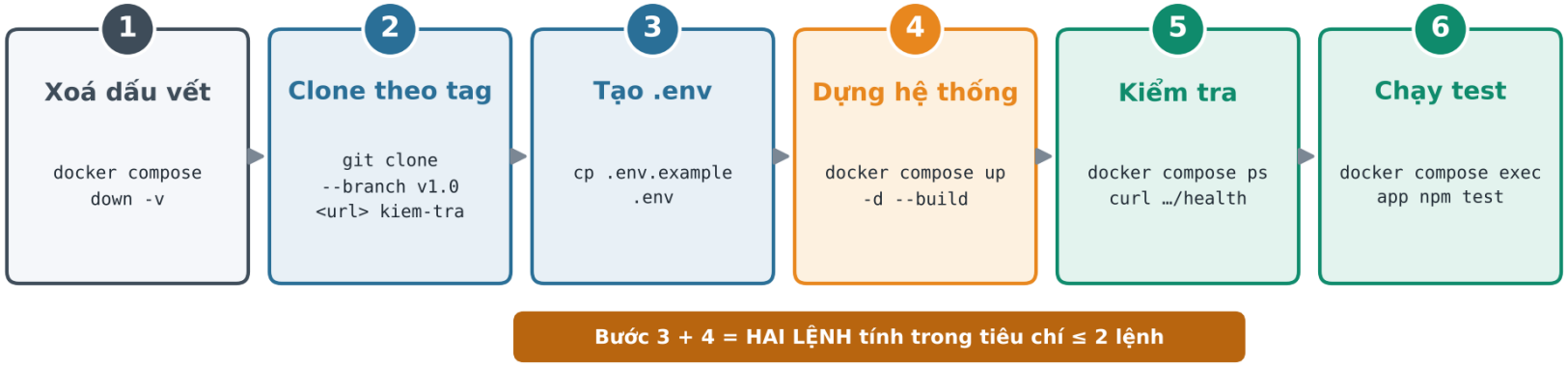
BỐN TỆP HAY BỊ NHẦM – MỖI TỆP MỘT VIỆC

.env, .env.example, .gitignore, .dockerignore

Tệp	Chứa gì	Commit?	Thiếu hoặc sai thì sao
.env	Giá trị THẬT dùng trên máy sinh viên: mật khẩu, khoá, chuỗi kết nối	**KHÔNG BAO GIỜ**	Lỡ commit: secret công khai vì repo PUBLIC → phải đổi khoá
.env.example	ĐỦ tên biến + giá trị MẪU chạy được cho môi trường thử; mỗi biến một dòng chú thích	Có	Thiếu một biến → app dừng ngay khi người chấm chạy (mất C1)
.gitignore	.env, node_modules/, .venv/, __pycache__/, tệp dữ liệu lớn, tệp mô hình	Có	Repo phình to, lộ .env, tệp mô hình vài trăm MB
.dockerignore	node_modules, .venv, .env, .git, dữ liệu thô, *.log	Có	Image chứa .env thật; node_modules của máy sinh viên đè lên bản cài trong image

Kiểm nhanh: **git check-ignore -v .env** phải in ra dòng .gitignore chặn nó; **git log --all --oneline -- .env** phải KHÔNG in dòng nào (.env chưa từng vào lịch sử — mục C7 của BT3).

Các bước kiểm chứng



Bước 3 + 4 = HAI LỆNH tính trong tiêu chí ≤ 2 lệnh

✓ ĐẠT (mục CỐT LÕI C1)

- Mọi dịch vụ ở trạng thái healthy
- /health trả {"status":"ok"}; có dữ liệu mẫu để thao tác
- Bộ test PASS bằng lệnh ghi trong Guide
- Không phải hỏi sinh viên thêm bước nào

✗ CHƯA ĐẠT — thường vì

- Thiếu biến trong .env.example → app dừng ngay
- Hệ thống lên nhưng trống trơn: quên dữ liệu mẫu
- app khởi động trước khi CSDL sẵn sàng
- Phải gõ thêm lệnh migrate/seed không có trong Guide

Đọc hình thế nào

- Clone theo **tag v1.0** vào thư mục trống — không dùng thư mục sinh viên đang làm việc.
- Làm **đúng** theo Deployment Guide, không suy đoán, không hỏi sinh viên.
- Phải hỏi sinh viên một bước → Deployment Guide chưa đạt.
- Chụp ảnh kết quả phép thử (ps healthy, /health, test PASS) cho **mục 7 của PDF**.

KHI PHÉP THỬ MÁY TRẮNG THẤT BẠI

Đọc triệu chứng — biết sửa ở đâu

Triệu chứng	Nguyên nhân thường gặp	Sửa thế nào
app khởi động rồi thoát: "Thiếu biến môi trường: JWT_SECRET"	.env.example thiếu biến mà máy sinh viên có trong .env	Bổ sung biến vào .env.example và bảng biến của Guide
ECONNREFUSED db:5432 ngay lúc khởi động	app chạy trước khi CSDL sẵn sàng	healthcheck cho db + depends_on condition: service_healthy
relation "tickets" does not exist	Volume cũ đã có dữ liệu nên initdb không chạy; hoặc tệp .sql nằm trong thư mục con	docker compose down -v; để tệp .sql ngang hàng trong db/init/
Hệ thống lên nhưng danh sách trống, không tạo được phiếu	Thiếu dữ liệu danh mục (trung tâm, kỹ thuật viên)	Thêm 002_seed_danh_muc.sql; Guide mục 4 ghi số bản ghi để đối chiếu
Bind for 0.0.0.0:3000 failed: port is already allocated	Cổng đã bị chương trình khác chiếm	Đổi APP_PORT trong .env; ghi vào mục xử lý sự cố của Guide
Test PASS trên máy sinh viên, FAIL trong container	Test phụ thuộc dữ liệu hoặc tệp chỉ có trên máy sinh viên	Test tự dựng dữ liệu (resetDb, fixture) — buổi 13

Quy trình gỡ bốn bước của buổi 11 vẫn áp dụng: đọc log trước (docker compose logs app --tail 50) → khoanh vùng → sửa nguyên nhân → chạy lại phép thử TỪ ĐẦU. Mỗi lỗi tìm được là một mục Bug Log tốt.

PHƯƠNG ÁN THAY THẾ VÀ KIỂM TRA THEO TRACK

Không dùng được Docker? Và “hệ thống đã lên” nghĩa là gì?

Phương án thay thế (đề BT3, mục 2.4)

- **Script dựng môi trường** (setup.sh, Makefile) — được chấp nhận nếu **đã thoả thuận trước** với giảng viên.
- Script phải chạy được trên môi trường sạch, **kiểm và báo lỗi rõ** khi thiếu phụ thuộc, ghi rõ phiên bản runtime.
- Ghi lý do chọn phương án này trong Deployment Guide — đó là một quyết định kỹ thuật có lập luận.
- Chỉ có hướng dẫn bằng lời → **KHÔNG đạt** thành phần 4.

[SE] và [AI] — dịch vụ chạy liên tục

- docker compose ps: mọi dịch vụ **healthy**.
- [SE] curl …:3000/health → {"status":"ok","db":"up"}; đăng nhập tài khoản demo ghi trong Guide.
- [AI] curl -X POST …:8000/predict với một mô tả mẫu → nhãn + độ tin cậy.
- [AI] tắt model: app vẫn tạo phiếu, chỉ thiếu gợi ý.

[DA] — pipeline chạy một lần

- docker compose run --rm pipeline → in báo cáo: số dòng đọc, nạp, bị loại kèm lý do.
- Chạy **lần thứ hai** → số dòng bảng đích **không đổi** (mục C8).
- Dashboard (nếu có) mở được và có biểu đồ từ dữ liệu vừa nạp.
- Guide ghi số dòng mong đợi để người chấm đối chiếu.

Mục 5 của Deployment Guide phải ghi **đúng các lệnh này kèm kết quả mong đợi cụ thể** — đó là thứ người chấm làm theo.

02

Biến môi trường và secret

Vì sao tách cấu hình · đọc và kiểm biến khi khởi động · bốn nơi không để secret · khi lỡ lộ

VÌ SAO TÁCH CẤU HÌNH RA KHỎI MÃ NGUỒN

Cùng một mã nguồn, nhiều nơi chạy, mỗi nơi một cấu hình

Mã nguồn — giống nhau ở mọi nơi

- Được commit, được chia sẻ, được chấm.
- Không chứa mật khẩu, khoá, địa chỉ máy chủ.
- Đổi môi trường **không** phải sửa mã và build lại.

Cấu hình — khác nhau ở mỗi nơi

- Máy sinh viên · máy giảng viên · bản deploy thử: mỗi nơi một DB_PASSWORD, một CORS_ORIGINS.
- Đọc từ **biến môi trường**, không ghi cứng.
- Danh sách biến được công bố trong .env.example và Deployment Guide.

Secret — một loại cấu hình đặc biệt

- Mật khẩu CSDL, JWT_SECRET, khoá API, khoá giả danh hoá [DA].
- Lộ ra là **người khác làm được việc nhân danh hệ thống**.
- Không bao giờ commit, không ghi log, không chụp vào ảnh.

Phép thử nhanh: nếu ngày mai sinh viên phải công khai repo cho cả lớp (BT3 bắt buộc PUBLIC), có dòng nào trong repo sinh viên phải xoá trước không? Có → dòng đó là secret đang nằm sai chỗ.

ĐỌC BIẾN MÔI TRƯỜNG ĐÚNG CÁCH

.env.example công bố biến, config.js kiểm biến

.env.example — commit, giá trị MẪU

```
# Sao chép:  cp .env.example .env
# KHÔNG commit .env. Đổi mọi giá trị
# khi triển khai thật.

APP_PORT=3000          # cổng mở ra máy chủ
DB_NAME=smartcrm
DB_USER=smartcrm
DB_PASSWORD=doi-mat-khau-nay
# >= 32 ký tự; sinh bằng:
#   openssl rand -hex 32
JWT_SECRET=thay-bang-chuoi-ngau-nhien-32-ky-tu
CORS_ORIGINS=http://localhost:3000
LOG_LEVEL=info
```

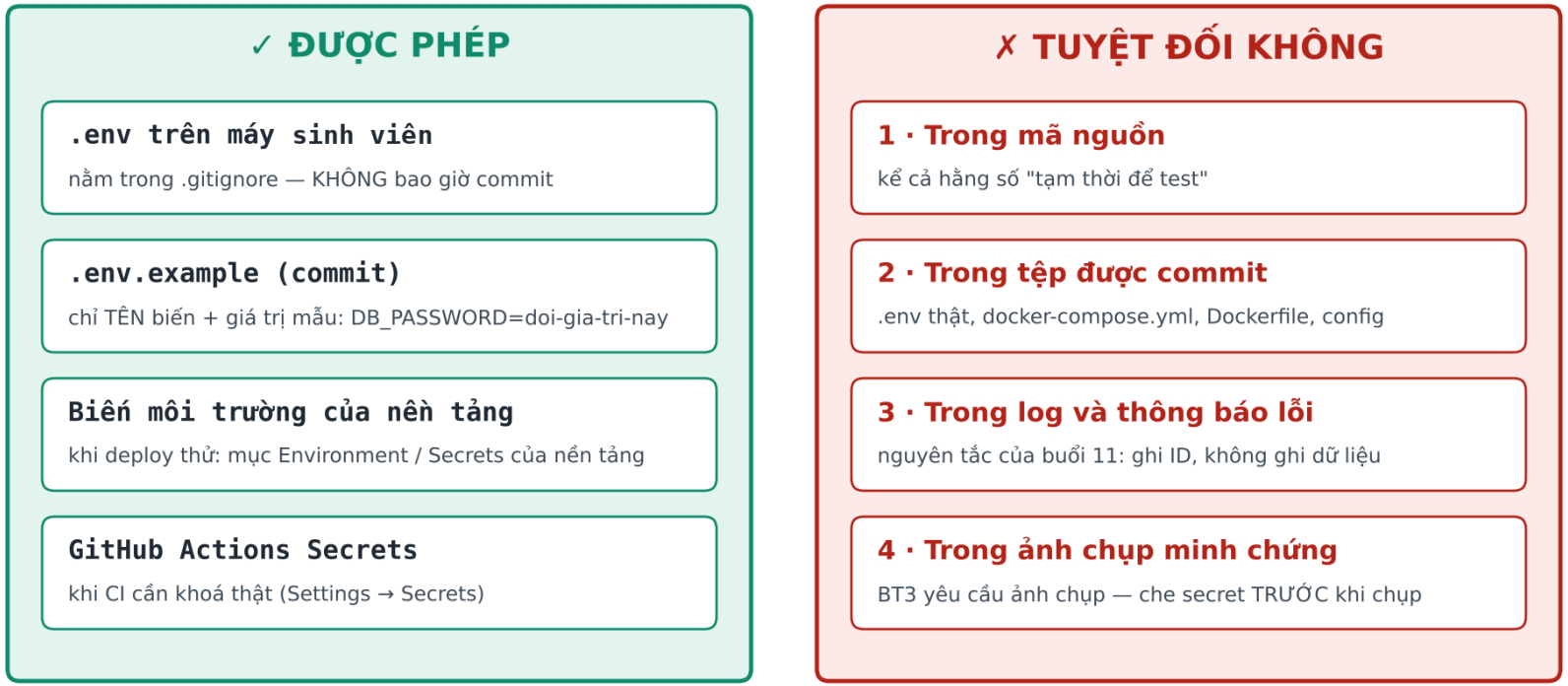
src/config.js — đọc MỘT lần, dừng sớm nếu thiếu

```
const BAT_BUOC = ['DB_HOST', 'DB_NAME', 'DB_USER',
                  'DB_PASSWORD', 'JWT_SECRET'];
const thieu = BAT_BUOC.filter(k => !process.env[k]);
if (thieu.length) {
  // In TÊN biến, không bao giờ in GIÁ TRỊ
  console.error('Thiếu biến môi trường: '
    + thieu.join(', ') + ' — xem .env.example');
  process.exit(1);
}
module.exports = {
  port: Number(process.env.PORT || 3000),
  db: { host: process.env.DB_HOST,
        database: process.env.DB_NAME,
        user: process.env.DB_USER,
        password: process.env.DB_PASSWORD },
  jwtSecret: process.env.JWT_SECRET,
};
```

Giá trị mẫu phải chạy được để hai lệnh của người chấm thành công; Guide ghi rõ phải đổi khi triển khai thật.

Dừng sớm với thông báo rõ tốt hơn chạy tiếp rồi lỗi khó hiểu ở chỗ khác. [DA][AI]: os.environ["TEN"] cũng báo KeyError ngay.

Bốn nơi tuyệt đối không được để secret



Repo BT3 là PUBLIC: một secret lọt vào lịch sử Git là đã công khai — phải ĐỔI KHOÁ, xoá ở commit sau không đủ.

Đọc hình thế nào

- Nơi thứ tư hay bị quên: **ảnh chụp minh chứng** — BT3 yêu cầu nhiều ảnh, che secret trước khi chụp.
- Docker image cũng là "tệp được chia sẻ": không COPY .env vào image.
- Deploy thử: đặt secret trong mục biến môi trường của nền tảng, không ghi vào repo.

NẾU LỠ ĐỂ LỘ SECRET

Ba bước theo đúng thứ tự — đổi khoá trước tiên

1

ĐỔI KHOÁ NGAY — trước cả việc sửa repo

Đổi mật khẩu CSDL, sinh JWT_SECRET mới, thu hồi khoá API cũ trên trang của dịch vụ. Repo PUBLIC: phải coi như khoá cũ **đã bị người khác đọc**.

2

Gỡ tệp khỏi theo dõi và chặn lại

git rm --cached .env → bổ sung .env vào .gitignore → commit. Xoá ở commit sau **KHÔNG xoá khỏi lịch sử** — ai clone cũng xem lại được commit cũ.

3

Kiểm hậu quả và ghi lại

Khoá dịch vụ trả phí (bản đồ, gửi tin nhắn): xem nhật ký sử dụng có bất thường không. Ghi việc đã xử lý vào tài liệu — đề BT3 mục 4.4 yêu cầu.

*Kiểm lịch sử trước khi nộp: **git log --all --oneline -- .env** (phải rỗng) và **git log -p --all | grep -inE "password|secret|api_key" | head**. Viết lại lịch sử Git (filter-repo) là kỹ thuật nâng cao — đổi khoá vẫn là bước bắt buộc.*

03 Deployment Guide

Viết cho người chưa biết gì về hệ thống · năm mục của đề BT3 · bằng tiếng Anh

DEPLOYMENT GUIDE – THÀNH PHẦN 5 CỦA BT3, TIẾNG ANH, 1–2 TRANG

Năm mục bắt buộc — và lỗi thường gặp ở mỗi mục

Mục	Nội dung tối thiểu	Lỗi thường gặp
1. Requirements (yêu cầu môi trường)	Hệ điều hành, phiên bản Docker/Compose hoặc runtime, dung lượng đĩa, cổng cần trống	Chỉ ghi "cần Docker", không ghi phiên bản và cổng
2. Setup steps (các bước dựng)	Đánh số; mỗi bước MỘT lệnh sao chép được; không có bước ngầm hiểu	Bỏ sót bước nạp dữ liệu mẫu → hệ thống lên nhưng trống trơn
3. Environment variables (biến môi trường)	Bảng: tên · ý nghĩa · giá trị mẫu · bắt buộc hay không	Chỉ ghi "xem .env.example", không giải thích biến nào làm gì
4. Sample data (dữ liệu mẫu)	Cách nạp; nạp xong có bao nhiêu bản ghi ở đâu để đối chiếu	Có dữ liệu mẫu nhưng không nói cách nạp, không nói đối chiếu thế nào
5. Verify it works (kiểm tra đã chạy đúng)	Lệnh hoặc bước cụ thể kèm KẾT QUẢ MONG ĐỢI cụ thể	Bỏ hẳn mục này — bị thiếu nhiều nhất, giảng viên phải tự đoán

Mục 5 phân biệt Guide viết theo trí nhớ với Guide viết sau khi **thật sự thử trên môi trường sạch**. Thêm một mục ngắn "Troubleshooting" (cổng bị chiếm, volume cũ) là điểm cộng về chất lượng — không bắt buộc.

VÍ DỤ — MỘT DEPLOYMENT GUIDE ĐỦ NĂM MỤC

docs/deployment-guide.md (rút gọn)

Viết bằng tiếng Anh — câu ngắn, lệnh sao chép được, kết quả mong đợi cụ thể

```
# Deployment Guide – Smart CRM, flow L2 (track SE), tag v1.0
## 1. Requirements
- Docker Engine 24+ and Docker Compose v2 (check: docker compose version)
- Git 2.30+, about 2 GB free disk, port 3000 free. No Node.js/PostgreSQL needed.
## 2. Setup steps
1. git clone --branch v1.0 https://github.com/<account>/<repo>.git && cd <repo>
2. cp .env.example .env
3. docker compose up -d --build        # first build takes about 2 minutes
## 3. Environment variables
| Name        | Meaning                              | Sample value     | Required |
| DB_PASSWORD | PostgreSQL password                  | doi-mat-khau-nay | yes      |
| JWT_SECRET  | Key for signing tokens (>= 32 chars) | (see example)    | yes      |
| APP_PORT    | Host port of the app                 | 3000             | no       |
## 4. Sample data
Loaded automatically from db/init/002_seed_danh_muc.sql on first start:
3 service centres, 6 technicians, 20 customers, 25 devices, 8 tickets.
Demo login: tiepnhan01 / Demo@2026 (role NHAN_VIEN_TIEP_NHAN).
## 5. Verify it works
docker compose ps                    -> app and db are "healthy"
curl http://localhost:3000/health    -> {"status":"ok","db":"up"}
docker compose exec app npm test     -> Tests: 12 passed, 12 total
```

Ghi nhớ

- Mỗi bước **một lệnh**, có thể sao chép nguyên văn.
- Mục 4 có **con số** để người chấm đối chiếu (8 phiếu, 20 khách).
- Mục 5: mỗi lệnh kèm **kết quả mong đợi** — chụp ảnh đúng các kết quả này cho mục 7 của PDF.
- Tài khoản demo là dữ liệu mẫu, không phải secret thật — mật khẩu trong CSDL vẫn phải băm.

Bốn thành phần của một cảnh báo — thiếu một là vô dụng

Thành phần	Phải ghi gì	Chưa đạt → Đạt
0. Cơ chế giám sát nền	Log có mã yêu cầu (request id) để lần theo; endpoint /health; thu số liệu: thời gian phản hồi, tỉ lệ lỗi 5xx, số kết nối CSDL	Chỉ có log rời rạc → log có mã yêu cầu + /health + vài chỉ số đo được
1. Đo cái gì	MỘT đại lượng cụ thể, đo ở đâu, bao lâu đo một lần	"Theo dõi hệ thống" → "tỉ lệ phản hồi 5xx của /api/tickets, mỗi 5 phút"
2. Ngưỡng bao nhiêu	Một CON SỐ, kèm khoảng thời gian để tránh báo nhầm	"Khi lỗi nhiều" → "> 2% trong 2 cửa sổ 5 phút liên tiếp"
3. Gửi cho ai	Một vai trò cụ thể và kênh nhận	"Báo cho admin" → "người vận hành trực ca, qua tin nhắn"
4. Người đó làm gì	Bước xử lý đầu tiên, viết sao cho người không tham gia phát triển làm theo được	(bỏ trống) → "xem log theo request id của các yêu cầu lỗi; nếu CSDL không phản hồi thì chạy docker compose restart db rồi kiểm /health"

Thành phần 4 là thành phần hay bị bỏ nhất và quan trọng nhất: cảnh báo mà người nhận không biết phải làm gì sẽ bị bỏ qua sau vài tuần — khi đó còn tệ hơn không có cảnh báo.

VÍ DỤ — MỤC MONITORING & ALERTS TRONG TÀI LIỆU BÀN GIAO

Hai cảnh báo cho luồng Smart CRM — viết bằng tiếng Anh

Thêm vào cuối Deployment Guide (hoặc README) — mỗi cảnh báo đủ bốn thành phần

```
## 6. Monitoring & alerts
Monitoring: every log line has a request id; GET /health checks app + db;
p95 latency and 5xx rate of /api/* are recorded every minute.

### ALERT-01  API errors
- Measure : share of 5xx responses on /api/tickets, 5-minute window
- Threshold: > 2% in 2 consecutive windows
- Notify  : on-duty operator (chat message)
- Action  : 1) open logs, filter by the request ids of failed calls
            2) if the db is unreachable: docker compose restart db,
               then curl /health must return {"db":"up"}
            3) still failing after 10 min -> escalate to the developer

### ALERT-02  Tickets about to miss the committed deadline (business)
- Measure : number of open tickets due within 4 hours, checked hourly
- Threshold: > 10, or 2x the same hour last week
- Notify  : service centre manager (e-mail)
- Action  : reassign tickets from the overdue list to free technicians
```

Ghi nhớ

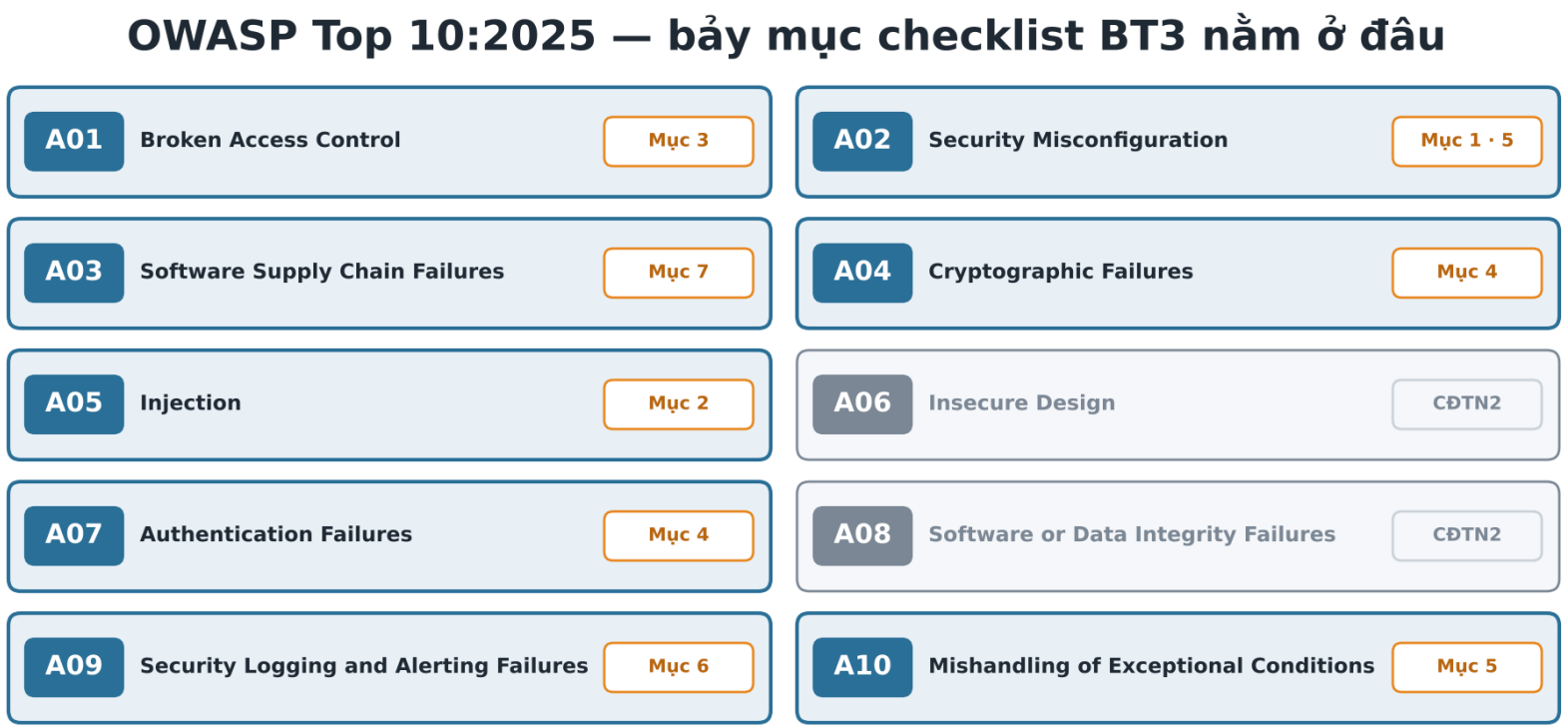
- Mỗi cảnh báo trả lời đủ bốn câu: đo gì · ngưỡng · gửi ai · làm gì.
- Ngưỡng có cả con số và khoảng thời gian.
- Phần Action đánh số bước, có lệnh sao chép được và kết quả mong đợi — như mục Setup steps.
- Nên có ít nhất một cảnh báo NGHIỆP VỤ, không chỉ cảnh báo kỹ thuật.
- BT3 không bắt buộc mục 6; câu 3c đề thi thì có.

04

Checklist bảo mật cơ bản

OWASP Top 10:2025 ở mức nhận biết · bảy mục tự đánh giá · mỗi mục một minh chứng chạy thật

Mười nhóm rủi ro — bảy mục checklist BT3 nằm ở đâu



Ô xanh: liên quan trực tiếp tới prototype và được tự kiểm qua checklist bảy mục · Ô xám: nhận biết, xử lý đầy đủ ở Chuyên đề tốt nghiệp 2.

Bản 2025 khác bản 2021: A03 Supply Chain và A10 Exceptional Conditions là hai nhóm mới; Misconfiguration lên hạng 2.

Đọc hình thế nào

- OWASP Top 10 là danh sách rủi ro phổ biến nhất của ứng dụng web, cập nhật vài năm một lần.
- Bản 2025 thêm **A03 Supply Chain** (phụ thuộc bên ngoài) và **A10 Exceptional Conditions** (xử lý lỗi).
- A01 Broken Access Control vẫn đứng đầu — cũng là lỗ hổng hay gặp nhất trong prototype sinh viên.

Mỗi mục: tự kiểm bằng gì, ghi minh chứng gì

Mục checklist (đề BT3)	OWASP	Tự kiểm bằng	Minh chứng ghi vào checklist
1. Không có secret nào trong repo	A02	git log --all --oneline -- .env; grep secret/password trong src/	Kết quả rỗng + ảnh chụp terminal
2. Dữ liệu vào được kiểm ở tầng ứng dụng	A05	Ba lệnh grep chống injection (tài liệu tự học A4); gửi dữ liệu sai	Test dữ liệu sai PASS (400/422); kết quả grep
3. Kiểm soát truy cập theo chủ sở hữu	A01	Đăng nhập khách A, sửa phiếu của khách B	Test case trả 403 PASS + ảnh
4. Mật khẩu, dữ liệu nhạy cảm băm/mã hoá	A04 · A07	SELECT password_hash FROM users LIMIT 1	Chuỗi dạng $2b$10$... (bcrypt), không đọc được
5. Thông báo lỗi không lộ cấu trúc trong	A10 · A02	Gửi JSON hỏng; tắt db rồi gọi API	Phản hồi không có stack, tên bảng, chuỗi kết nối
6. Nhật ký không ghi secret, dữ liệu cá nhân	A09	docker compose logs app | grep -iE "password|token|0[0-9]{9}"	Kết quả rỗng + ảnh chụp
7. Phụ thuộc ghim phiên bản, đã kiểm lỗ hổng	A03	npm audit --omit=dev · pip-audit -r requirements.txt	Kết quả audit; cảnh báo còn lại kèm kế hoạch

*Mỗi mục ghi ĐÃ LÀM / CHƯA LÀM / KHÔNG ÁP DỤNG kèm MỘT dòng minh chứng. Ghi "CHƯA LÀM" trung thực **không bị trừ điểm**; ghi "ĐÃ LÀM" mà không có minh chứng thì bị trừ ở tiêu chí 4 và ảnh hưởng CLO5.*

VÍ DỤ — MỤC 2 VÀ A05 INJECTION

Ghép chuỗi SQL và tham số hoá: một dòng khác nhau

× Ghép chuỗi — nhập ' OR '1'='1 là đọc được mọi khách

```
// Express
const sql = `SELECT * FROM customers
             WHERE phone = '${req.query.sdt}'`;
const kq = await db.query(sql);

# Python
cur.execute(f"SELECT * FROM customers "
            f"WHERE phone = '{sdt}'")

-- Kẻ tấn công gửi:  sdt = ' OR '1'='1
-- Câu SQL thành:
--   ... WHERE phone = '' OR '1'='1'
--   -> trả về TOÀN BỘ khách hàng
```

✓ Tham số hoá — dữ liệu không bao giờ thành mã SQL

```
// Express + pg
const kq = await db.query(
  'SELECT id, full_name FROM customers WHERE phone = $1',
  [req.query.sdt]);

# Python + psycopg
cur.execute(
  "SELECT id, full_name FROM customers WHERE phone = %s",
  (sdt,))

-- Thư viện gửi câu lệnh và dữ liệu TÁCH RIÊNG:
-- ' OR '1'='1 chỉ là một chuỗi số điện thoại
-- không khớp -> 0 dòng.
-- Kèm: chỉ SELECT cột cần, không SELECT *
```

Dấu hiệu cần tìm: dấu nháy + dấu cộng hoặc ${...} / f"...{...}" bên trong câu SQL. Lệnh grep ở tài liệu tự học tìm đúng mẫu này.

Test kèm theo (loại dữ liệu sai): gửi sdt = ' OR '1'='1 → mong đợi 0 kết quả hoặc 400, không phải danh sách khách. Ghi vào Test Report.

VÍ DỤ — MỘT CHECKLIST ĐƯỢC ĐIỂM

docs/checklist-bao-mat.md: trạng thái + một dòng minh chứng

Ba trạng thái đều hợp lệ — miễn là đúng sự thật và có minh chứng

```
## 1. Không có secret nào trong repo — ĐÃ LÀM
Minh chứng: git log --all --oneline -- .env -> rỗng;
            grep -rniE "password|secret|api_key" src/ -> rỗng.
Ảnh: docs/screenshots/sec-1.png

## 3. Kiểm soát truy cập theo chủ sở hữu — ĐÃ LÀM
Minh chứng: TC-09 khách A sửa phiếu của khách B -> 403, PASS.
Ảnh: docs/screenshots/TC-09.png

## 4. Mật khẩu được băm — ĐÃ LÀM
Minh chứng: SELECT password_hash FROM users LIMIT 1 -> $2b$10$Qe... (bcrypt)

## 7. Phụ thuộc ghim phiên bản, đã kiểm lỗ hổng — CHƯA LÀM xong
Minh chứng: package-lock.json đã commit. npm audit còn 2 cảnh báo moderate
            ở gói chỉ dùng khi test. Kế hoạch: nâng phiên bản sau khi nộp BT3.

## 4. (track AI) Mật khẩu được băm — KHÔNG ÁP DỤNG
Lý do: prototype AI không có đăng nhập, không lưu mật khẩu.
```

Ghi nhớ

- "CHƯA LÀM" kèm kế hoạch là câu trả lời **trung thực và được chấp nhận**.
- "KHÔNG ÁP DỤNG" phải có **một dòng lý do**.
- Ảnh chụp đặt trong docs/screenshots/, đặt tên theo mục hoặc theo mã test.
- Giảng viên **đối chiếu** với repo: gây một lỗi, kiểm secret, thử quyền truy cập (bước 7 quy trình chấm).

RANH GIỚI PHẠM VI VÀ TRUNG THỰC

Làm đủ bảy việc — không làm thừa, không khai khống

CĐTN1 yêu cầu (bắt buộc ở BT3)

- Bảy mục checklist, tự kiểm bằng lệnh và test.
- Nhận biết mười nhóm OWASP Top 10:2025.
- Ít nhất một test kiểm quyền truy cập [SE] / một phép đo chất lượng [DA] / test chống rò rỉ [AI].

Thuộc Chuyên đề tốt nghiệp 2

- Kiểm thử xâm nhập, rà soát bảo mật đầy đủ.
- Mô hình hoá mối đe doạ, thiết kế an toàn (A06).
- Vault quản lý secret, luân chuyển khoá tự động.
- Vận hành hệ thống giám sát đầy đủ, ứng phó sự cố (viết cảnh báo bốn thành phần thì thuộc CĐTN1).

Nguyên tắc trung thực

- "Chưa làm" trung thực **không bị trừ điểm**.
- "Đã làm" không có minh chứng → **bị trừ** tiêu chí 4, ảnh hưởng CLO5.
- Một ảnh chụp kết quả lệnh có giá trị hơn mười dòng mô tả.

Lời khuyên từ đề BT3: điền checklist bằng cách CHẠY THỬ — thường phát hiện được **một bug Critical thật**, vừa được điểm checklist vừa có thêm một mục Bug Log tốt.

05

Tài liệu kỹ thuật theo track và nộp Bài tập 3

Thành phần 6 · nội dung nộp · điểm cộng deploy thử

PHẦN CHUYÊN NGÀNH – THÀNH PHẦN 6 CỦA BT3, TIẾNG ANH

Tài liệu kỹ thuật riêng theo chuyên ngành

Tài liệu tự học buổi 15 (song ngữ) có mẫu điền sẵn cho từng track. Quy tắc chung: mô tả đúng thứ hệ thống THẬT SỰ làm — không mô tả endpoint, cột hay chỉ số chưa có.

SE API docs

- OpenAPI/Swagger, Postman collection đã xuất tệp, hoặc Markdown
- Đủ **MỌI** endpoint đang có
- Mỗi endpoint **bảy mục**: phương thức + đường dẫn · mô tả · xác thực · tham số · phản hồi thành công · bảng mã lỗi · giới hạn
- Ví dụ request/response **THẬT** chép từ lần chạy

DA Pipeline doc + Data dictionary

- Pipeline doc **sáu mục**: sơ đồ pha · nguồn · đích và grain · quy tắc làm sạch kèm số dòng loại · lịch chạy và chạy bù · hạn chế đã biết
- Data dictionary: **mọi cột** của mọi bảng đích
- Mỗi cột sáu trường, có **phân loại dữ liệu** và cách xử lý bảo mật

AI Model card

- **Mười mục** theo mẫu tài liệu tự học
- Mục 3: ≥ 3 trường hợp **KHÔNG nên dùng**
- Mục 8: thiên lệch **đo thật** theo ≥ 2 cách nhóm
- Mục 9: ≥ 2 rủi ro, mỗi rủi ro kèm biện pháp giảm thiểu cụ thể

NỘI DUNG NỘP BÀI TẬP 3 · 10% ĐIỂM HỌC PHẦN

Nộp gì cho Bài tập 3 — và nộp thế nào

1 · Hình thức và thời hạn

- Nộp qua **VLU E-learning**: **1 file PDF + link GitHub repo có tag v1.0**.
- Tên file: **BT3_<MSSV>_<Họ tên không dấu>.pdf** — một file duy nhất.
- Hạn: **trước khi kết thúc buổi 16**.
- Nộp bù: **tối đa 3 ngày**, trừ **10% điểm mỗi ngày** trễ.
- Quá 3 ngày: 0 điểm thành phần.
- Repo để **PUBLIC** — giảng viên không nhận lời mời collaborator; vì vậy không có .env hay secret.

2 · Sáu thành phần trong repo (tag v1.0)

- **TP1 – Test Plan** rút gọn, bốn mục.
- **TP2 – Test Report:** 10–12 test case đủ **tám cột**, đủ năm loại, có kết quả CHẠY THẬT.
- **TP3 – Bug Log:** ≥ 3 bug (≥ 1 Critical/Major), mỗi bug bảy dòng, link commit mở được.
- **TP4 – Gói đóng gói** dựng lại được bằng ≤ 2 lệnh từ thư mục trống.
- **TP5 – Deployment Guide** tiếng Anh (năm mục) + checklist bảo mật bảy mục.
- **TP6 – Tài liệu kỹ thuật** tiếng Anh: API docs [SE] · Pipeline doc + Data dictionary [DA] · Model card [AI].

3 · Cấu trúc file PDF

- **Trang bìa:** link repo + tag v1.0, track, mã luồng; link deploy thử nếu có.
- **Mục 1** Test Plan · **Mục 2** Test Report.
- **Mục 3** Bug Log · **Mục 4** Deployment Guide.
- **Mục 5** Checklist bảo mật · **Mục 6** Tài liệu kỹ thuật.
- **Mục 7 ẢNH CHỤP minh chứng:** test PASS · hệ thống lên được · phép thử máy trắng.
- **Phụ lục (bắt buộc):** bảng khai báo sử dụng công cụ AI.

ẢNH CHỤP KẾT QUẢ CHẠY THẬT là minh chứng bắt buộc (mục 7 của PDF): thấy rõ lệnh đã chạy và kết quả — test PASS, docker compose ps healthy, phép thử máy trắng. Test case hay mục nào không có ảnh chụp thì tính là **không có minh chứng** — chụp ngay khi chạy, đừng để tới lúc nộp.

ĐIỂM CỘNG TỰ CHỌN – DEPLOY THỬ LÊN NỀN TẢNG CÓ GÓI MIỄN PHÍ

Chỉ làm khi sáu thành phần bắt buộc đã xong

Mức	Điều kiện	Điểm cộng vào BT3
Không làm	—	0
Deploy được, mở xem được	Đường dẫn công khai mở được tại thời điểm chấm; ghi trên trang bìa PDF	+0,5
Deploy đầy đủ	Mở được, biến môi trường cấu hình đúng trên nền tảng, VÀ Deployment Guide có thêm một mục ngắn ghi các bước đã làm để deploy	+1,0

Ba lưu ý: (1) **đừng làm** nếu sáu thành phần chưa xong — mỗi học kỳ đều có sinh viên mải deploy mà bỏ dở tài liệu; (2) **tuyệt đối không** đưa secret thật lên repo hay vào image — dùng mục biến môi trường của nền tảng; (3) gói miễn phí thường **tự ngủ** — mở lại đường dẫn ngay trước khi giảng viên chấm.

TỰ KIỂM TRA NHANH

Năm câu hỏi trước khi rời lớp

1. **Vì sao khoá phiên bản thư viện (mức 2) vẫn chưa đủ cho Bài tập 3?**
   Gợi ý: runtime và CSDL trên máy người chấm thì sao?

2. **Bỏ condition: service_healthy khỏi depends_on thì chuyện gì có thể xảy ra khi người chấm chạy?**
   Gợi ý: container “đã khởi động” khác “đã sẵn sàng” thế nào?

3. **Tuần trước sinh viên lỡ commit .env có mật khẩu thật. Xoá tệp ở commit hôm nay có đủ không? Làm gì trước tiên?**
   Gợi ý: repo PUBLIC và lịch sử Git.

4. **Sinh viên chưa chạy npm audit / pip-audit. Mục 7 của checklist ghi thế nào để không mất điểm?**
   Gợi ý: ba trạng thái hợp lệ.

5. **Deployment Guide thiếu mục nào thì giảng viên phải tự đoán hệ thống đã chạy đúng hay chưa?**
   Gợi ý: mục bị thiếu nhiều nhất theo đề BT3.

KẾT THÚC BUỔI HỌC

Chốt buổi 15 và chuẩn bị cho buổi 16 — ★ NỘP BÀI TẬP 3

Tự học, tự nghiên cứu

- Viết Dockerfile, docker-compose.yml, .dockerignore, db/init/ cho prototype của sinh viên.
- **Chạy phép thử máy trắng NGAY tuần này**: down -v → clone theo tag thử vào thư mục trống → up → ps → test; chụp ảnh kết quả.
- Viết Deployment Guide tiếng Anh đủ năm mục.
- Điền checklist bảo mật bảy mục bằng cách chạy thử, mỗi mục có minh chứng.
- Đọc tài liệu tự học theo track; viết tài liệu kỹ thuật của track.
- Chọn một xu hướng công nghệ của track để trình bày 5 phút ở **buổi 18** (đăng ký, nộp slide PDF trên VLU E-learning trước giờ học).

Buổi 16 (thực hành) — Workshop đóng gói prototype & tài liệu kỹ thuật

- Hoàn thiện gói đóng gói; giảng viên chạy phép thử máy trắng tại lớp.
- Hoàn thiện Deployment Guide, checklist bảo mật, tài liệu kỹ thuật theo track.
- (Khuyến khích) deploy thử lên nền tảng có gói miễn phí.
- ★ NỘP BÀI TẬP 3 trước khi kết thúc buổi: 1 file PDF + link repo PUBLIC có tag v1.0.